TD6 — TP N°3: CNNs en EuroSAT

Ariel Alzogaray Flores, Lucas Rehmann, Josefina Deserti Gotta

Noviembre de 2026

Índice

1. Introducción **3**

2. Desarrollo de la solución **3**
2.1. Modelado de la CNN customizada 3
2.2. Entrenamiento y optimización de hiperparámetros 4
2.3. Evaluación del modelo 5
2.4. *Transfer learning* 7
2.5. *Disclaimer* sobre uso de IA 9

3. Conclusión y consideraciones finales **9**

1. Introducción

El objetivo del presente trabajo consiste en entender y explorar de primera mano los conceptos de redes neuronales convolucionales (CNN) y *transfer learning* a través de una aplicación particular en el campo del análisis de imágenes. Más concretamente, se desea implementar dos CNN independientes para resolver un problema de clasificación de imágenes satelitales. La primera será diseñada y entrenada desde cero, mientras que la segunda será una CNN previamente entrenada para la clasificación de imágenes. Esto motivará la comparación entre ambos enfoques, donde cabe observar la *accuracy*, velocidad y *performance* general de cada modelo.

2. Desarrollo de la solución

2.1. Modelado de la CNN customizada

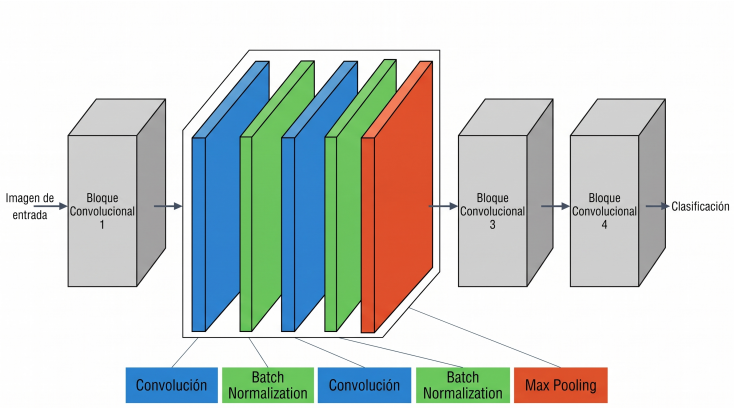


Figura 1: Estuctura de la CNN customizada.

La CNN propuesta cuenta con 4 bloques convolucionales, los cuales se componen de la siguiente manera:

1. Una convolución de tamaño 3×3, con *stride* y *padding* igual a 1, que preserva el tamaño de la imagen original.

2. Una capa de *Batch Normalization* que normaliza los datos de salida, lo cual ayuda a mejorar la convergencia del método SGD y evita una competencia injusta entre valores de magnitudes dispares.

3. Una segunda convolución, con las mismas características que la primera.

4. Una segunda capa de *Batch Normalization*.

5. Una capa de *Max Pooling* con un kernel de tamaño 2×2 y un *stride* igual a 2. Se eligió este tipo de *pooling* para priorizar la conservación de la detección de los patrones.

2.2. Entrenamiento y optimización de hiperparámetros

El criterio de optimización elegido (la *loss function*) es la *Cross Entropy*. El algoritmo de optimización implementado es el *Mini-Batch SGD* con *batches* de 64 imágenes, combinado con un *momentum* de 0,9 para poder escapar al efecto zig-zag y a las mesetas.

El modelo fue entrenado con 20 épocas, llegando a un *accuracy* del 96,69 % para el conjunto de validación. Asimismo, para controlar que no se produzca ningún tipo de efecto indeseado sobre los gradientes, se registró su evolución por cada *batch* de entrenamiento. Fenómenos tales como *vanishing* o *exploding gradients* pueden ralentizar e incluso impedir la convergencia del algoritmo.

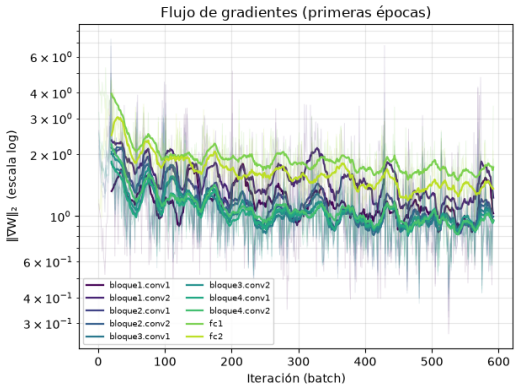


Figura 2: Evolución de los gradientes a lo largo del algoritmo de optimización.

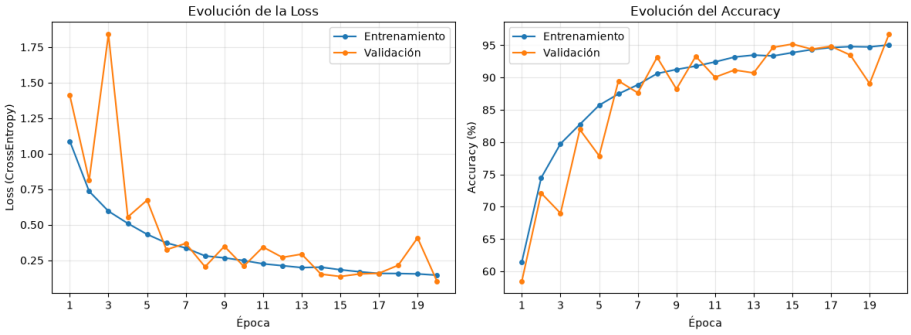


Figura 3: Resultados del entrenamiento de la CNN customizada. Sobre el eje Y, los valores de *loss* y *accuracy* obtenidos, respectivamente. Sobre el eje X, la época alcanzada.

2.3. Evaluación del modelo

Clase	Precision	Recall	F1-score	Support
AnnualCrop	0.934	0.973	0.953	450
Forest	0.991	0.984	0.988	450
HerbaceousVegetation	0.903	0.956	0.929	450
Highway	0.973	0.957	0.965	375
Industrial	0.978	0.968	0.973	375
Pasture	0.956	0.943	0.950	300
PermanentCrop	0.964	0.931	0.947	375
Residential	0.976	0.987	0.981	450
River	0.980	0.925	0.952	375
SeaLake	0.991	0.993	0.992	450
Macro avg	0.965	0.962	0.963	4050
Weighted avg	0.964	0.964	0.964	4050

Cuadro 1: Métricas por clase sobre el conjunto de test.

Finalizado el entrenamiento, el modelo fue evaluado sobre el conjunto de test, compuesto por 4050 imágenes no vistas durante el entrenamiento y la validación. Se obtuvo un *accuracy* del 96,37 % y un *F1-score* macro de 0,9630. La similitud del valor de ambas métricas indica que el modelo generaliza de forma efectiva sin incurrir en *overfitting*.

Se reporta el promedio macro además del *accuracy* global del dado que el conjunto esta desbalanceado (las clases cuentan con entre 300 y 450 instancias), de modo tal que el promedio macro pondera por igual a todas las categorías y evita ocultar la *performance* segregada por clase. Como ya dijimos, ambas métricas son fuertemente similares por lo que se puede decir que el modelo no tiene una preferencia sobre una clase particular.

Se observa que la *performance* es bastante alta para todas las clases con una brecha sutil de a lo sumo unas pocas décimas. Las categoría con mejor *performance* fue `SeaLake` (con 0,992), mientras que la peor fue `HerbaceousVegetation` (con 0,929).

La matriz de confusión permite identificar dónde yacen los errores del modelo.

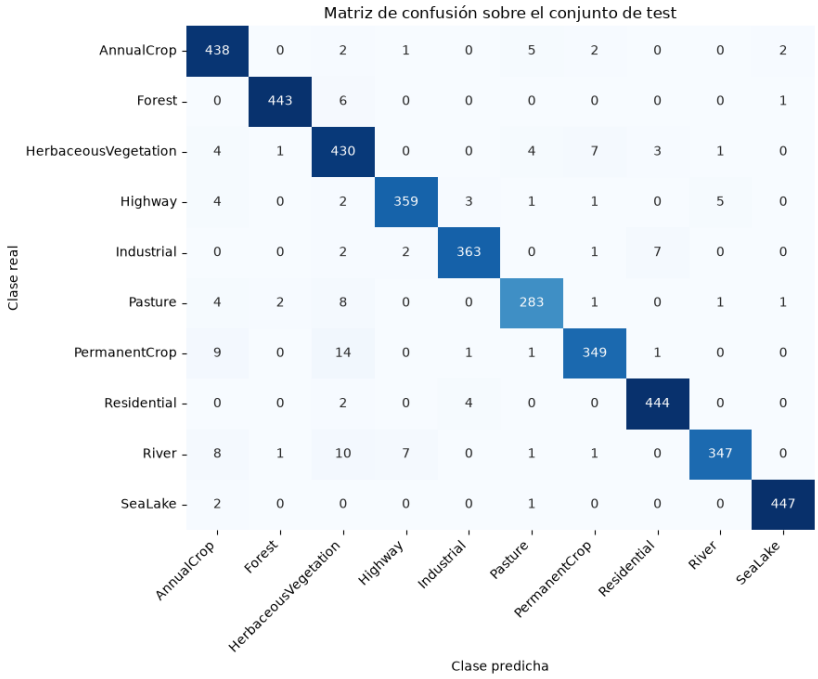


Figura 4: Matriz de confusión sobre el conjunto de test. Las filas corresponden a la clase real y las columnas a la clase predicha por el modelo.

En particular, es preciso observar cómo estos se concentran en las clases vegetales.

Por otra parte, el modelo no parece confundir agua con vegetación ni zonas construidas con bosques. De este modo, se deduce que el modelo sabe distinguir las estructuras correspondientes a cada clase, dejando un margen de mejora más orientado hacia los detalles.

Como complemento al análisis previo, se proyectaron las representaciones internas del modelo (vectores de 256 dimensiones que produce la penúltima capa) sobre el plano (dos dimensiones) mediante t-SNE. La visualización resultó consistente con los hallazgos ya mencionados en la matriz de confusión.

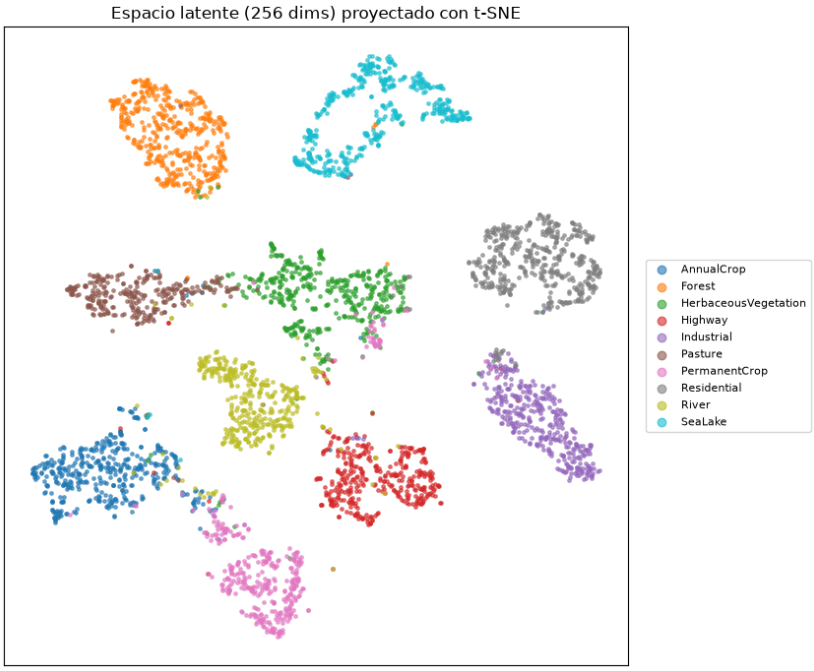


Figura 5: Proyección mediante t-SNE del espacio latente de 256 dimensiones correspondiente a la penúltima capa de la red, coloreada según la clase real.

Primero que nada, `Forest`, `SeaLake`, `Residential` e `Industrial` forma agrupamientos compactos y claramente aislados del resto, mientras que las clases vegetales ocupan una región central contigua en la que las fronteras de los *clusters* son algo más difusas (hay un solapamiento sutil de clases para algunos puntos). Esto refuerza la idea de que el problema no radica en el clasificador final sino en que las representaciones de estas clases son intrínsecamente cercanas entre sí.

Cabe aclarar que t-SNE preserva la estructura de los datos mas no las distancias entre diferentes agrupamientos ni sus tamaños relativos. Por ende, esta visualización sólo permite analizar el agrupamiento de puntos de una misma clase y el solapamiento de las mismas.

2.4. *Transfer learning*

El modelo elegido para comparar es ResNet18. Este modelo preentrenado trabaja con imágenes de 224×224, por lo que será necesario redimensionar el

conjunto de imágenes de EuroSAT, que tienen tamaño 64×64. Asimismo, también será necesario normalizar sus canales de color de acuerdo al entrenamiento previo de ResNet18. Esto se debe a que la red podría estar acostumbrada a trabajar con ciertos tipos de patrones en el tono, brillo o saturación (entre otros) que pueden perder visibilidad si se le proporcionan imágenes en otros espectros del color.

Por otra parte, se conserva el mismo esquema de *data augmentation* y los mismos índices de partición, de modo que ambos modelos evalúan sobre exactamente las mismas imágenes.

Además, se mantienen la totalidad de los parámetros preentrenados salvo por aquellos del último bloque residual (`layer4`), y se reemplaza la capa clasificadora por una capa densa de 10 salidas. La intuición detrás de esto es que las primeras capas capturan patrones generales que son independientes del problema a resolver, mientras que las últimas son más específicas para clasificar las clases en ImageNet por lo que debieron ser ajustadas a EuroSat. Para este último bloque se emplea una tasa de aprendizaje diez veces menor que para la capa clasificadora (0,001 frente a 0,01), ya que parte de esos pesos ya ajustados y conviene no degradarlos, mientras que la capa final se inicializa desde cero. En ambos casos se mantuvo el *Stochastic Gradient Descent* con momento 0,9.

El entrenamiento consistió en 2 épocas, ya que cada época demoró 883 segundos. Aún así, el modelo alcanzó un $94,44\,\%$ de accuracy en validación tras una sola época, lo que muestra que partir de representaciones ya aprendidas acelera notablemente la convergencia en términos de épocas.

	CNN propia	ResNet18
Accuracy en test	96,37 %	95,83 %
F1 macro	0,9630	0,9566
Parámetros entrenables	2.224.618	8.398.858
Épocas de entrenamiento	20	2

Cuadro 2: Comparación de ambos enfoques sobre el conjunto de test.

Sobre test, ResNet18 obtuvo un accuracy de $95,83\,\%$ y un F1 macro de 0,9566, levemente por debajo de la arquitectura customizada. El hecho de que el modelo preentrenado no haya superado al modelo construído desde cero se explica principalmente por la cantidad de épocas que fue entrenado (dos contra veinte) y la dimensión del conjunto de entrenamiento, pues este es lo suficientemente grande como para entrenar el modelo desde cero sin dificultad, mientras que el *transfer learning* opera mejor sobre un conjunto de datos chicos. Sin embargo, es destacable la rapidez con que el modelo preentrenado se acercó a los resultados de la red customizada. Es indudable que, de haber entrenado con una mayor cantidad de épocas, se hubieran superado con creces los resultados.

2.5. *Disclaimer* sobre uso de IA

Vale la pena aclarar que, durante el desarrollo del trabajo, se utilizaron herramientas de inteligencia artificial generativa para elaborar los gráficos pertinentes con `matplotlib`. Asimismo, el dibujo ilustrativo de la CNN customizada también fue obtenido de este modo.

El resto del código presentado es pura y exclusivamente de elaboración propia.

3. Conclusión y consideraciones finales

La *performance* de las CNN para la clasificación multiclase de imágenes ha demostrado ser muy buena, tanto para la customizada como para la preentrenada. Si bien ResNet18 es mucho más densa y requiere más tiempo de cómputo, a la larga el *trade off* se compensa pues la cantidad de épocas requerida para lograr un buen *accuracy* es mucho menor. Tampoco se debe despreciar el rendimiento de la primera red, que logró un 96,37 % de *accuracy* en el conjunto de testeo. En síntesis, ambos modelos probaron ser aptos para resolver la problemática propuesta. Decidir cuál de los dos implementar es fundamentalmente preguntarse si se quiere priorizar la velocidad de cómputo o la precisión de la solución.

Es preciso destacar que, durante la realización del presente trabajo, las instancias de entrenamiento y evaluación fueron ejecutadas por CPU. Una ejecución por GPU lograría mejoras notables en cuanto a velocidad. Por lo tanto, los tiempos reportados deben ser considerados en términos relativos y no absolutos. Asimismo, el entrenamiento de la red preentrenada se vio afectado por las mismas limitaciones de *hardware*, por lo que la cantidad de épocas implementadas es menor de la que convendría probar.